Formulas we use

Server Delay Analyzer · finding the minimum delay of a server. n = number of worker threads, b = batch size, D = delay in milliseconds.

#	Formula	Type	What it does
1	**Delay model (our formula)** $D(n, b) = D_0 + A/n + B \cdot n + C/b + E \cdot b + k \cdot n \cdot b$	**Calculus**	Gives the delay for n worker threads and batch size b. Each term is one cause: queue (A/n), crowding (B·n), setup (C/b), waiting for the batch (E·b), clash (k·n·b), base (D_0).
2	**Least squares fit** minimise $\| W (X \cdot \theta - d) \|^2$, with $\theta \geq 0$ X columns: 1, 1/n, n, 1/b, b, n·b $\theta = (D_0, A, B, C, E, k)$	**Linear algebra**	Finds the constants A, B, C, E, k, D_0 that best match the measured delays d. The formula is linear in these constants, so it is a matrix least-squares problem. $\theta \geq 0$ keeps every cost positive (non-negative least squares).
3	**Weights** $W = \text{diag}(1/d_1, 1/d_2, \ldots)$	**Linear algebra**	Makes the fit minimise relative (percentage) error, so one very slow reading cannot pull the whole fit.
4	**Fit quality** typical error = median(|predicted − measured| / measured) R^2 = 1 − (weighted residual sum of squares) / (weighted total sum of squares)	**Linear algebra**	Tells how well the formula matches the data. Good if typical error ≤ 10%, fair if ≤ 25%.
5	**Gradient (first partial derivatives)** $\partial D/\partial n = -A/n^2 + B + k \cdot b$ $\partial D/\partial b = -C/b^2 + E + k \cdot n$	**Calculus**	The slope of the delay in each direction.
6	**Condition for the minimum** $\nabla D = 0$, i.e. $\partial D/\partial n = 0$ and $\partial D/\partial b = 0$	**Calculus**	At the lowest delay both slopes are zero. Solving these two equations gives the best setting.
7	**Closed form when k = 0 (starting guess)** $n^* = \sqrt{(A/B)}$, $b^* = \sqrt{(C/E)}$	**Calculus**	With no clash term the two equations separate and solve by hand. We use this as the first guess.
8	**Hessian (second partial derivatives)** $H = \begin{bmatrix} 2A/n^3 , k \\ k , 2C/b^3 \end{bmatrix}$	**Calculus**	The curvature of the delay surface (how the slopes change).
9	**Newton's method step** solve $H \cdot \Delta = \nabla D$, then $(n, b)_{new} = (n, b) - t \cdot \Delta$	**Both**	Solves the two equations of step 6 when $k \neq 0$. Each step solves a 2×2 linear system (linear algebra) built from derivatives (calculus). It is the 2-variable version of $x_{new} = x - f'(x)/f''(x)$.
10	**Step check (backtracking)** accept t if $D(\text{new}) \leq D(\text{old}) - 0.0001 \cdot t \cdot (\nabla D \cdot \Delta)$, else halve t	**Calculus**	Makes sure every Newton step really lowers the delay and keeps n and b positive.

#	Formula	Type	What it does
11	**Eigenvalue test** $\det(H - \lambda I) = 0 \to \lambda_1, \lambda_2$; both $\lambda > 0 \Rightarrow$ minimum	**Linear algebra**	Proves the point found is a true minimum (lowest point), not a maximum or a saddle.
12	**Second derivative test (same check, calculus form)** $D_{nn} > 0$ and $D_{nn} \cdot D_{bb} - (D_{nb})^2 > 0$, with $D_{nb} = k$	**Calculus**	The classroom version of step 11: same answer, written with second derivatives.
13	**Log-scale (cosh) identity** with $u = \ln n$: $A/n + B \cdot n = 2\sqrt{(AB)} \cdot \cosh(u - u^*)$, $u^* = \frac{1}{2} \ln(A/B)$	**Calculus**	Shows the delay is a symmetric bowl on a doubling scale (1, 2, 4, 8 ...), which is why we measure and zoom on that scale (Log-Scale Bowl Search).
14	**Best whole-number setting** pick integers (i, j) near (n*, b*) with the smallest D(i, j)	**Calculus**	You cannot run 9.67 threads, so we check the whole numbers around the optimum.

How they fit together

1. Measure delays at many settings. **2.** Fit the formula with least squares (steps 2–4, linear algebra). **3.** Set the gradient to zero (steps 5–6, calculus). **4.** Solve it with Newton's method, starting from the k = 0 guess (steps 7–10). **5.** Prove it is a minimum with the Hessian eigenvalues (steps 11–12). **6.** Round to whole numbers and confirm by running it (step 14).

Our verified result (default case)

Constants	A = 400, B = 4, C = 90, E = 2.5, k = 0.05
Best setting	n* = 9.673458 threads, b* = 5.492191
Lowest delay	D* = 112.817896 ms
Hessian eigenvalues	0.8721 and 1.0982 (both positive, so it is a minimum)
Best whole numbers	10 threads × batches of 5 = 113.00 ms (only 0.16% above the minimum)
Independent check	verify.py (NumPy, SciPy, SymPy) gives the same numbers

Type key: purple = linear algebra, pink = calculus, green = both.